Seminar Report

MD-Bench CUDA Port

MuCoSim Seminar

Author: Maximilian Gaul

Date: February 27, 2022

Instructor: Dr. Jan Eitzinger

Supervisor: Rafael Ravedutti

Contents

1 Project Description **3**
1.1 Application Info . . . 3
1.2 Testsystem . . . 5
1.3 Software Environment . . . 6
1.4 How to build software . . . 6
1.5 Testcase description . . . 7
1.6 How to run software . . . 7

2 Scaling Runs **9**
2.1 CUDA Programming Model . . . 9
2.2 CUDA Kernels . . . 10
2.3 NVIDIA Tools . . . 10
2.4 GPU Baseline . . . 11
2.5 Improvement I: Pinnend memory . . . 12
2.6 Improvement II: Allocate and copy memory only once . . . 14
2.7 Improvement III: Reduce neighbour list copies . . . 16
2.8 Improvement IV: Force memory layout . . . 17
2.9 Improvement V: Force integration on GPU . . . 18
2.10 Improvement VI: Neighbour list memory layout . . . 20
2.11 Final performance comparisons . . . 21

3 Detailed kernel measurements **23**
3.1 Resource Utilization . . . 23
3.2 FLOPs/s . . . 24
3.3 Memory Transfers . . . 24

4 Conclusion **26**

Project Description

Application Info

MD-Bench is a molecular dynamics application which calculates interactions among particles and how these affect their motion. The simulation system consists of

- number of atoms (with initial state such as position & velocity)
- boundary conditions for 3D space

The force of each atom is based on its accumulated interactions with neighboring atoms. In particular, MD-Bench uses the *Lennard-Jones* potential to model the potential energy among pairs of particles. The model is able to represent attractive as well as repulsive energies.

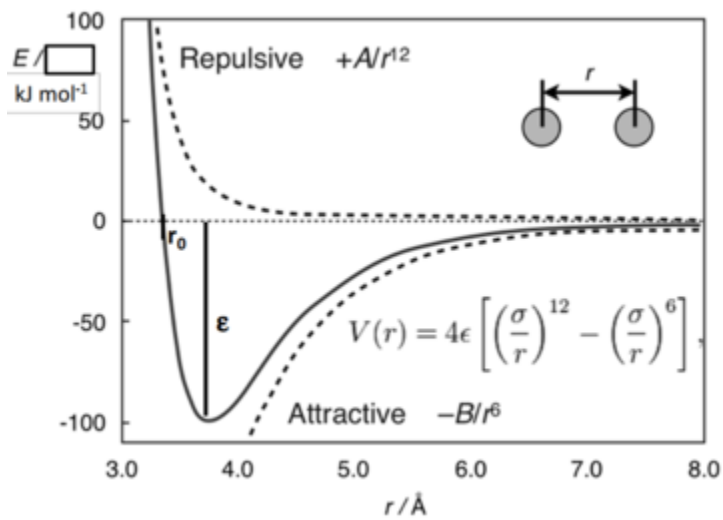


Figure 1.1: Lennard-Jones potential with the distance between two atoms in Ångström on the x-axis

In figure 1.1 the potential energy in relation to the distance between two atoms is depicted. The formula of the potential consists of three parameters, namely:

- r: distance between two interacting atoms
- ϵ: dispersion energy
- σ: distance at which the particle-potential V is zero

The figure shows that the Lennard-Jones potential is a simplified model but still describes the essential aspects of particle dynamics. Furthermore, particles repel each other at close, attract at medium and have negligible interactions at large distances.

MD-Bench is publicly available on GitHub via the URL `https://github.com/RRZE-HPC/MD-Bench`.

The main focus of this report is to port the main functions in *force.c* to the CUDA programming model and evaluate the performance with respect to the adapted code, which is available under the branch *mucosim-cuda*.

Calculating the Lennard-Jones potential, updating the atom forces as well as positions is done in a main-loop for each time step:

```
for t in range(timesteps):
    for atom in atoms:
        neighbors = neighbor.neighbors[atom]
        force = 0.0
        for neighbor in neighbors:
            radius = calc_radius(...)
            if radius < close_enough:
                force += calc_force(...)
        forces[atom] += force
    update_velocity(atoms)
    update_position(atoms)
    if t % 20 == 0:
        reneighbour_atoms(atoms)
```

Figure 1.2: Pseudocode of the main-loop of MD-Bench

Testsystem

Most of the upcomming performance measurements were performed on:

- Host/Clustername: tg060-tg06b (tinyGPU)
- Cluster Info URL: `https://hpc.fau.de/systems-services/systems-documentation-instructions/clusters/tinygpu-cluster/`
- CPU type: 2x Intel Xeon Gold 6134 ("Skylake") @ 3.2 GHz
- GPU type: 4x NVIDIA RTX 2080 Ti (11 GB memory)
- Memory: 96GB RAM

Towards the end there are metric evaluations which were performed on:

- Host/Clustername: tg090-tg097 (tinyGPU)
- Cluster Info URL: `https://hpc.fau.de/systems-services/systems-documentation-instructions/clusters/tinygpu-cluster/`
- CPU type: 2x AMD Rome 7662 @ 2.0 GHz
- GPU type: 4x NVIDIA A100 SXM4/Nvlink (40GB memory)
- Memory: 512GB RAM

To access either one of the nodes, one have to first connect to `woody.rrze.fau.de` and then execute

- srun.tinygpu -N 1 -n 1 –gres=gpu:rtx2080ti:1 –pty /bin/bash -l
- srun.tinygpu -N 1 -n 1 –partition=a100 –gres=gpu:a100:1 –pty /bin/bash -l

respectively.

Software Environment

The software is compiled with

- Cuda compilation tools, release 11.2, V11.2.152
- Build cuda_11.2.r11.2/compiler.29618528_0

on the Operating System Ubuntu 20.04.3 LTS.

How to build software

After connecting to woody.rrze.fau.de, the following commands need to be executed to clone to repository

```
foo@bar:~$ git clone https://github.com/RRZE-HPC/MD-Bench↘
    /tree/mucosim_cuda
foo@bar:~$ cd MD-Bench
foo@bar:MD-Bench$ git checkout mucosim_cuda
```

Figure 1.3: Cloning the repository

Then, the TAG-variable in config.mk needs to be changed to TAG ?= NVCC to use the Nvidia CUDA compiler. Compiler flags can be adjusted in include_NVCC.mk. For compiling the code, one has to first connect to a GPU cluster, in this case, either *tinyGPU.tg060-tg06b* or *tinyGPU.tg090-tg097* and then execute:

```
foo@bar:MD-Bench$ module load likwid cuda
foo@bar:MD-Bench$ make
```

Figure 1.4: Compiling the code

This should produce an executable MDBench-NVCC.

Testcase description

This report will focus on the standard configuration of MD-Bench, this means *131072* atoms, double precision and *Lennard-Jones* potential for the force field. Only the amount of time steps is modified. For performance tests, 2000 time steps on a RTX 2080 Ti are used. Due to the high measurement time, this value has been reduced to 200 for the metric evaluations, which are done on an A100.

How to run software

MD-Bench has the following command line parameters:

```
foo@bar:MD-Bench$ ./MDBench-NVCC -h
MD Bench: A minimalistic re-implementation of miniMD
-----------------------------------------------------
-f <string>:            force field (lj or eam), default lj
-i <string>:            input file for EAM
-n / --nsteps <int>:    set number of timesteps for ↘
   simulation
-nx/-ny/-nz <int>:      set linear dimension of systembox ↘
   in x/y/z direction
--freq <real>:          processor frequency (GHz)
--vtk <string>:         VTK file for visualization
-----------------------------------------------------
```

Figure 1.5: MD-Bench command line parameters

Running the previously generated executable should produce the following output

```
foo@bar:MD-Bench$ ./MDBench-NVCC
step     temp            pressure
0        1.440000e+00    1.215639e+00
100      8.200895e-01    6.923143e-01
200      7.961495e-01    6.721043e-01
---------------------------------------------------------
Force field: lj
Data layout for positions: AoS
Using double precision floating point.
---------------------------------------------------------
System: 131072 atoms 47265 ghost atoms, Steps: 200
TOTAL 6.83s FORCE 0.73s NEIGH 3.07s REST 3.04s
---------------------------------------------------------
Performance: 3.84 million atom updates per second
```

Figure 1.6: MD-Bench output

Scaling Runs

CUDA Programming Model

In order to scale the MD-Bench code efficiently on Nvidia GPUs, knowledge about the CUDA programming model is necessary.

The basic execution size is a *thread* which is executed by one CUDA core. Typical state-of-the art GPUs have 4352 (RTX 2080 Ti) and up to 6912 (A100) CUDA cores.

Threads are grouped together as *blocks*, these are then executed on Streaming Multiprocessors (*SM*). Resources on an SM are allocated on a *warp*-size level (multiple of 32 threads and up to 1024) and dispatched to execution units of the GPU.

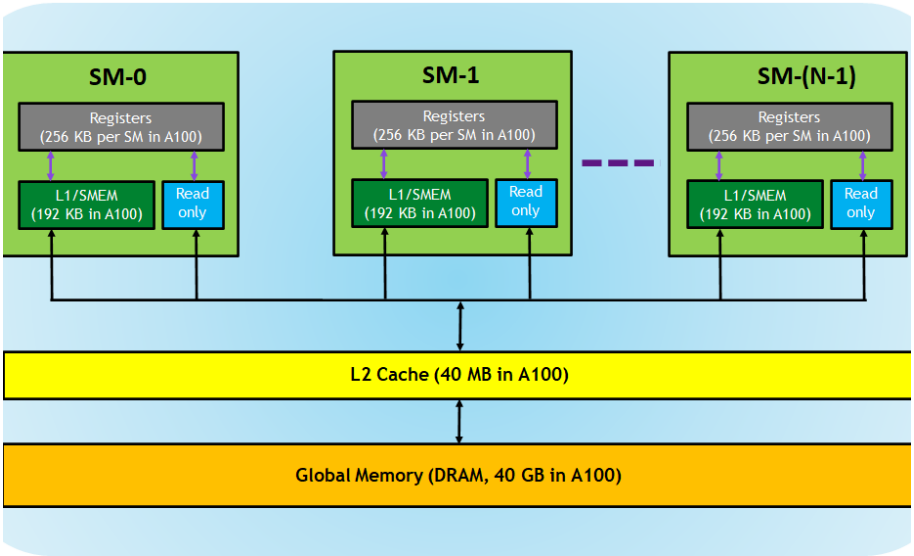


Figure 2.1: Memory hierarchy in GPUs

Figure 2.1 shows the memory architectur of Nvidia GPUs such as the A100. It consists of

- Registers: Owned by a thread, not visible to other threads
- L1/Shared memory (SMEM): Each SM has a versatile memory which can be used as L1 cache or as shared memory among all threads in a CUDA block. Size ranges from 64KB per SM (RTX 2080 Ti) to 192KB per SM (A100).
- Read-only memory: Each SM has an instruction cache as well as other kinds of memory which can be used for graphics applications.
- L2 cache: Shared across all SMs which means that every CUDA block can access this memory. Typical sizes are 5.5MB (RTX 2080 Ti) and up to 40MB (A100).

CUDA Kernels

Parallelization of force calculation on available CUDA cores is best done via the number of atoms. For 131072 atoms, this means there are 131072 threads and 131702 / num_threads_per_block number of blocks, where performance is evaluated for num_threads_per_block $\in \{1, 2, 4, 8, 16, 32, 64\}$. Each thread is going to execute the same *CUDA kernel* function in parallel on the GPU. In the context of MD-Bench, the kernel function calculates the 3D force for a single atom with respect to its neighbour-atoms.

Before the execution can happen, memory has to be copied from host (CPU) to device (GPU). This means the neighbour list as well as position data for each atom and Lennard-Jones potential parameters. The kernel-code writes the resulting forces into GPU memory which is copied back to the host after calculation is finished in order to proceed with other functions that are still running on CPU.

Inside the kernel function there is a precise way to tell which thread is currently executed. The CUDA programming model supplies the parameters blockIdx.x, blockDim.x and threadIdx.x. A combination of these parameters can be used as an index into the atom- and force-arrays.

NVIDIA Tools

NVIDIA supplies a range of tools to evaluate performance of code running on the GPU. nvprof is able to record memory transfers from and to the device. This helps with evaluating memory bottlenecks.

Furthermore, *NVIDIA Nsigh Compute* is available as a command line tool ncu and for the desktop. It captures a vide variety of performance parameters, some of them will be displayed in the metric evaluation section.

GPU Baseline

For an initial performance baseline, the MD-Bench computeForce function is refactored *as-is* to support force calculation in parallel using a CUDA kernel.

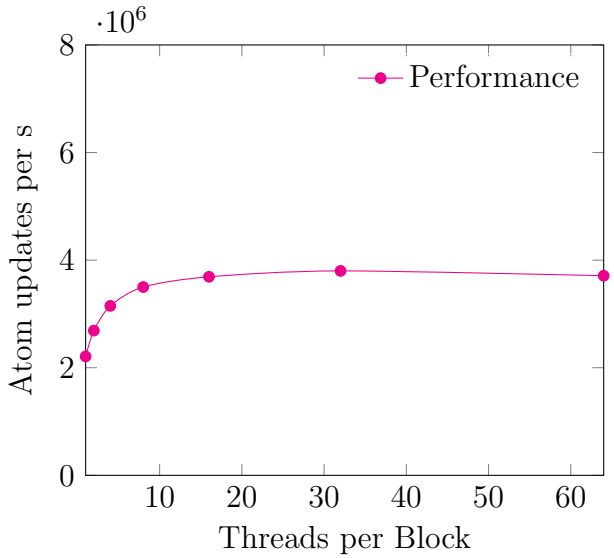


Figure 2.2: GPU baseline performance, RTX 2080 Ti

Using nvprof ––print–gpu–trace ./MDBench–NVCC yields an overview of memory transfers between host and device:

Duration	Size	Throughput	SrcMemType	DstMemType	Name	Type
899.68us	4.1199MB	4.4719GB/s	Pageable	Device	[CUDA memcpy HtoD]	atom.x/y/z
87.904us	1.0000MB	11.109GB/s	Pageable	Device	[CUDA memcpy HtoD]	atom.fx
87.616us	1.0000MB	11.146GB/s	Pageable	Device	[CUDA memcpy HtoD]	atom.fy
87.808us	1.0000MB	11.122GB/s	Pageable	Device	[CUDA memcpy HtoD]	atom.fz
60.640us	703.13KB	11.058GB/s	Pageable	Device	[CUDA memcpy HtoD]	atom.type
896ns	128B	136.24MB/s	Pageable	Device	[CUDA memcpy HtoD]	LJ ϵ
864ns	128B	141.29MB/s	Pageable	Device	[CUDA memcpy HtoD]	LJ σ
864ns	128B	141.29MB/s	Pageable	Device	[CUDA memcpy HtoD]	threshold
11.534ms	50.000MB	4.2335GB/s	Pageable	Device	[CUDA memcpy HtoD]	neighbour
45.248us	512.00KB	10.791GB/s	Pageable	Device	[CUDA memcpy HtoD]	numneigh
19.917ms	-	-	-	-	calcforce() [132]	-
80.896us	1.0000MB	12.072GB/s	Device	Pageable	[CUDA memcpy DtoH]	atom.fx
80.736us	1.0000MB	12.096GB/s	Device	Pageable	[CUDA memcpy DtoH]	atom.fy
80.480us	1.0000MB	12.134GB/s	Device	Pageable	[CUDA memcpy DtoH]	atom.fz

Table 2.1: Memory transfers for GPU baseline

Table 2.1 shows that the performance depicted in figure 2.2 is severely limited due to memory bottlenecks. Because of that, the main goal is to remove as many memory transfers as possible.

Improvement I: Pinnend memory

As a first performance improvement, memory allocations on the host are changed from regular malloc and free to their respective CUDA versions cudaMallocHost and cudaFreeHost.

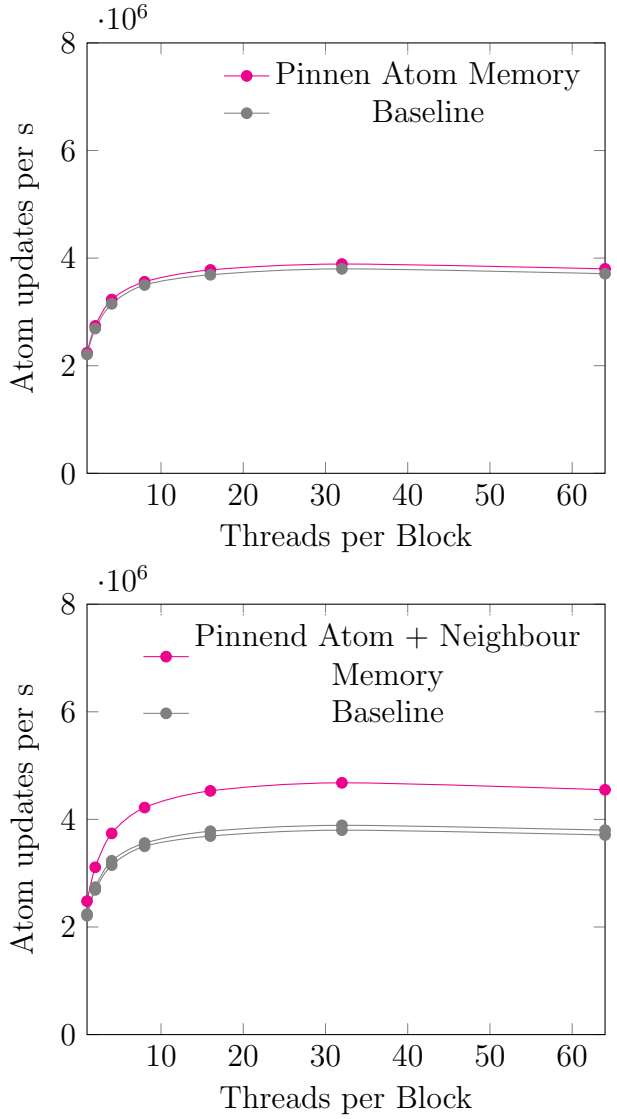


Figure 2.3: Improvement I: Pinnend memory

cudaMallocHost allocates memory on the host that is page-locked an accessible to the device. This means, this memory block is not available for paging and can therefore achieve higher data rates and less access delay.

As shown in figure 2.3, this yields a performance improvement of 23% compared to baseline if all memory is allocated this way, meaning atom as well as neighbour list.

Duration	Size	Throughput	SrcMemType	DstMemType	Name	Type
355.54us	4.1199MB	11.316GB/s	Pinned	Device	[CUDA memcpy HtoD]	atom.x/y/z
88.069us	1.0000MB	11.089GB/s	Pinned	Device	[CUDA memcpy HtoD]	atom.fx
87.972us	1.0000MB	11.101GB/s	Pinned	Device	[CUDA memcpy HtoD]	atom.fy
87.844us	1.0000MB	11.117GB/s	Pinned	Device	[CUDA memcpy HtoD]	atom.fz
61.187us	703.13KB	10.959GB/s	Pinned	Device	[CUDA memcpy HtoD]	atom.type
832ns	128B	146.72MB/s	Pinned	Device	[CUDA memcpy HtoD]	LJ ϵ
833ns	128B	146.54MB/s	Pinned	Device	[CUDA memcpy HtoD]	LJ σ
832ns	128B	146.72MB/s	Pinned	Device	[CUDA memcpy HtoD]	threshold
4.2854ms	50.000MB	11.394GB/s	Pinned	Device	[CUDA memcpy HtoD]	neighbour
45.443us	512.00KB	10.745GB/s	Pinned	Device	[CUDA memcpy HtoD]	numneigh
19.880ms	-	-	-	-	calcforce() [294]	-
81.285us	1.0000MB	12.014GB/s	Device	Pinned	[CUDA memcpy DtoH]	atom.fx
80.932us	1.0000MB	12.066GB/s	Device	Pinned	[CUDA memcpy DtoH]	atom.fy
80.868us	1.0000MB	12.076GB/s	Device	Pinned	[CUDA memcpy DtoH]	atom.fz

Table 2.2: Memory transfers after Improvement I

Table 2.2 clearly shows the improved memory throughput with pinnend memory for the atom position, neighbour list and Lennard-Jones parameters.

Improvement II: Allocate and copy memory only once

Instead of allocating memory in every call of computeForce, memory on the GPU is allocated only once during MD-Bench initialization.

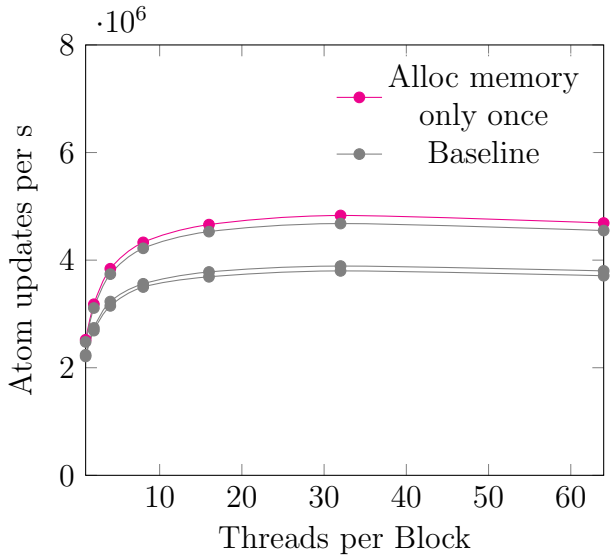


Figure 2.4: Improvement II: Allocate memory only once

Furthermore, the force arrays where the computed values for each atom will be stored after the GPU kernel is finished executing can be zeroed directly on the GPU instead of copying a zeroed array from host to device. This will replace three memory transfers with three memset.

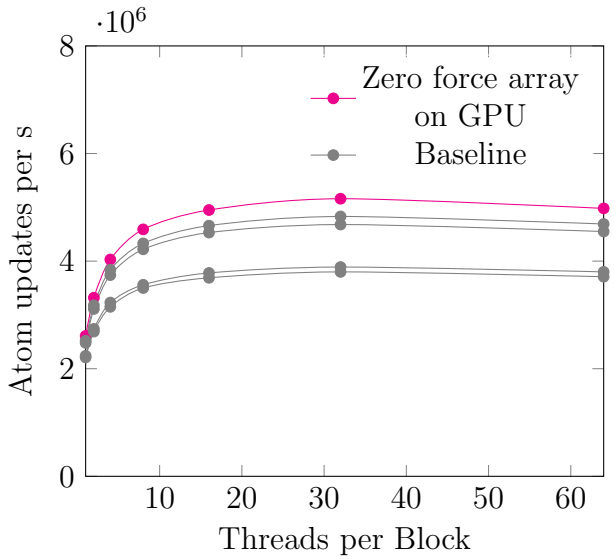


Figure 2.5: Improvement II: Zero force array on GPU

The Lennard-Jones parameters and the force cutoff radius only need to be copied once to GPU because they don't change throughout the calculation. This means, memory transfers can be reduced by three and performance potentially improved.

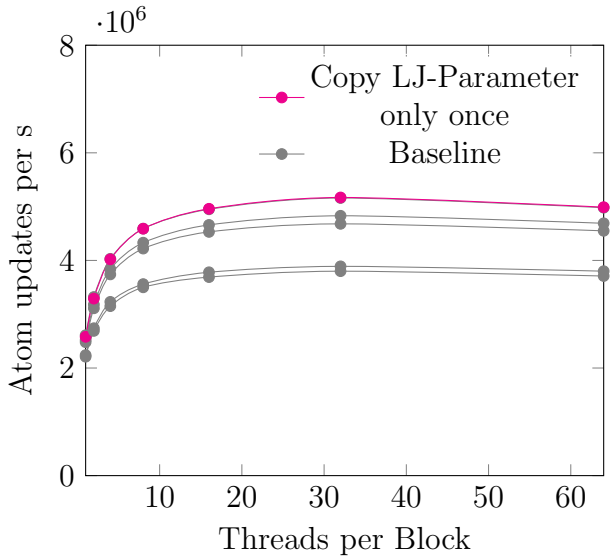


Figure 2.6: Improvement II: Copy LJ-Parameters only once

Duration	Size	Throughput	SrcMemType	DstMemType	Name	Type
354.67us	4.1199MB	11.344GB/s	Pinned	Device	[CUDA memcpy HtoD]	atom.x/y/z
3.2640us	1.0000MB	299.19GB/s	Device	-	[CUDA memset]	atom.fx
3.2650us	1.0000MB	299.10GB/s	Device	-	[CUDA memset]	atom.fy
2.8800us	1.0000MB	339.08GB/s	Device	-	[CUDA memset]	atom.fz
4.2804ms	50.000MB	11.407GB/s	Pinned	Device	[CUDA memcpy HtoD]	neighbour
45.058us	512.00KB	10.837GB/s	Pinned	Device	[CUDA memcpy HtoD]	numneigh
1.5061ms	-	-	-	-	calcforce() [308]	-
80.771us	1.0000MB	12.091GB/s	Device	Pinned	[CUDA memcpy DtoH]	atom.fx
80.899us	1.0000MB	12.071GB/s	Device	Pinned	[CUDA memcpy DtoH]	atom.fy
80.803us	1.0000MB	12.086GB/s	Device	Pinned	[CUDA memcpy DtoH]	atom.fz

Table 2.3: Memory transfers after Improvement II

Figures 2.4 and 2.5 show improved performance because they avoid large memory transfers as depicted in table 2.3. However, figure 2.6 doesn't show a significant improvement over figure 2.5 which is due to the small size of the Lennard-Jones parameters.

In general, performance improved by around 10% in this section.

Improvement III: Reduce neighbour list copies

Every 20th time step, the neighbour list is updated because the atoms have moved away from their original position due to changes in force and velocity. This means, it is not necessary to copy neighbour related data in every time step such that 19 out of 20 memory transfers can be safed.

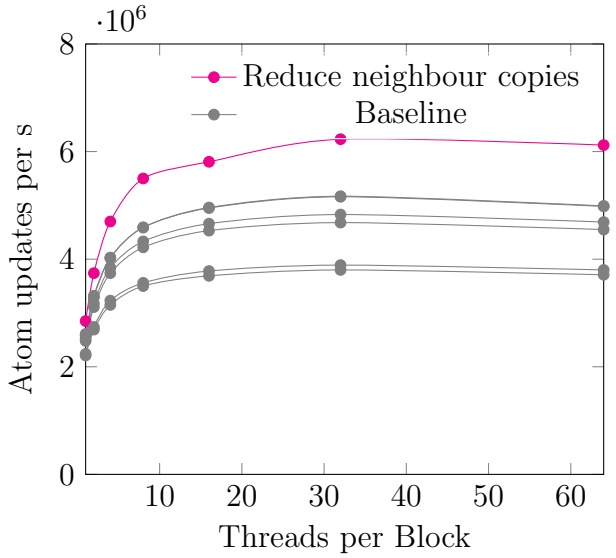


Figure 2.7: Improvement III: Reduce neighbour list copies

Figure 2.7 shows quite a large improvement in performance of 21% compared to the previous baseline.

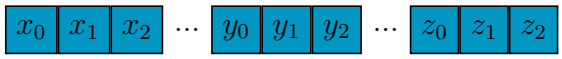
Duration	Size	Throughput	SrcMemType	DstMemType	Name	Type
354.67us	4.1199MB	11.344GB/s	Pinned	Device	[CUDA memcpy HtoD]	atom.x/y/z
3.3600us	1.0000MB	290.64GB/s	Device	-	[CUDA memset]	atom.fx
3.2960us	1.0000MB	296.29GB/s	Device	-	[CUDA memset]	atom.fy
2.9760us	1.0000MB	328.15GB/s	Device	-	[CUDA memset]	atom.fz
1.4924ms	-	-	-	-	calcforce() [306]	-
80.740us	1.0000MB	12.095GB/s	Device	Pinned	[CUDA memcpy DtoH]	atom.fx
80.996us	1.0000MB	12.057GB/s	Device	Pinned	[CUDA memcpy DtoH]	atom.fy
80.836us	1.0000MB	12.081GB/s	Device	Pinned	[CUDA memcpy DtoH]	atom.fz

Table 2.4: Memory transfers after Improvement III

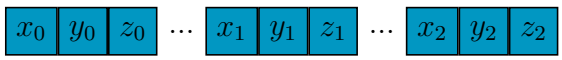
As expected, table 2.4 shows that 50.5 MB of memory transfers were spared.

Improvement IV: Force memory layout

The default implementation for the force memory is Structure of Arrays (*SoA*). Assuming the memory was an infinitely long 1D array, then the force data would be laying in memory like this



Changing the force memory layout to Array of Structures (*AoS*) would look like this



By doing this, force data of multiple atoms can be loaded into L1-Cache at once while the original SoA-approach would probably take multiple depending on the amount of atoms.

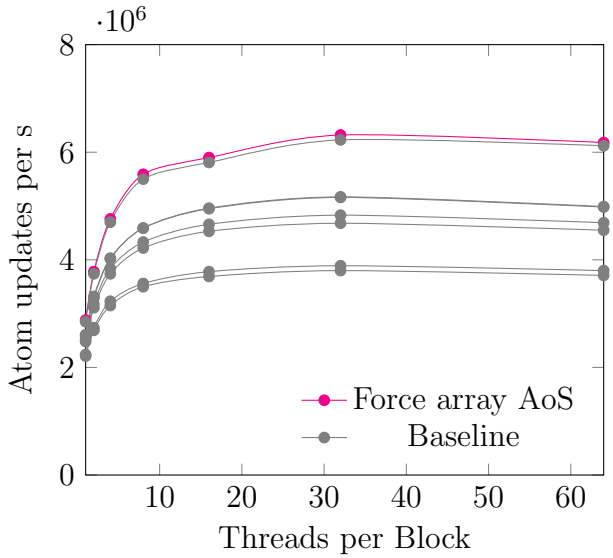


Figure 2.8: Improvement IV: Force memory layout

Duration	Size	Throughput	SrcMemType	DstMemType	Name	Type
355.01us	4.1199MB	11.333GB/s	Pinned	Device	[CUDA memcpy HtoD]	atom.x/y/z
5.0240us	3.0000MB	583.14GB/s	Device	-	[CUDA memset]	atom.fx/fy/fz
1.5084ms	-	-	-	-	calcforce() [288]	-
242.34us	3.0000MB	12.089GB/s	Device	Pinned	[CUDA memcpy DtoH]	atom.fx/fy/fz

Table 2.5: Memory transfers after Improvement IV

Figure 2.8 only shows a marginal improvement of around 2% which can be explained by the fact that only little time was saved for the memory transfers. As depcited in table 2.5, around 4.5µs were spared for the force array memset and hardly anything for the force copy to host.

Improvement V: Force integration on GPU

The two functions initial_integrate and final_integrate are summing over the forces for each atom and calculate the change in velocity and position. They can be parallelized on an atom level similar to the computeForce function which makes them relatively easy to port to the GPU.

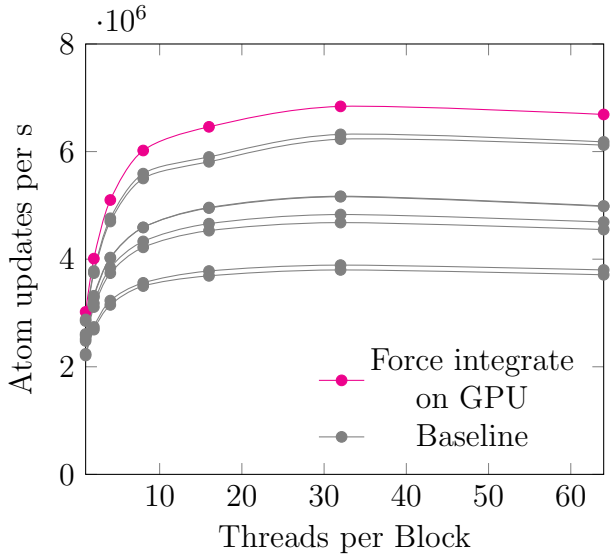


Figure 2.9: Improvement V: Force integration on GPU

Because the integration needs force data as input and returns velocity and position, these arrays have to be copied from host to device and vice versa, respectively.

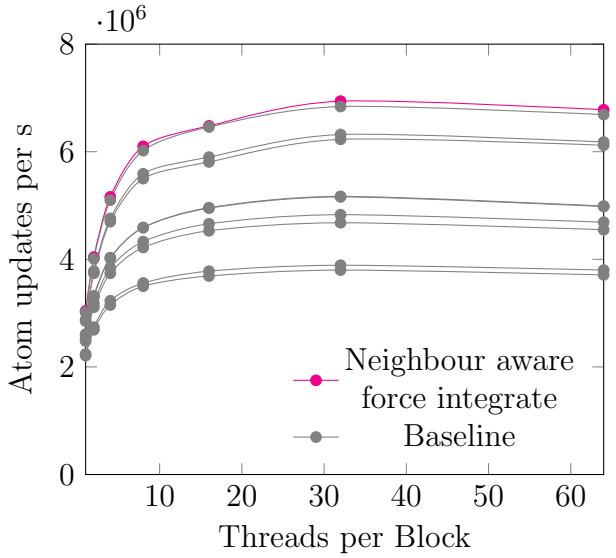


Figure 2.10: Improvement V: Neighbour aware force integration on GPU

Figure 2.10 depicts the performance if those memory transfers are reduced to the number of times a neighbour list update is happening which is every 20th iteration.

Because the force data is now completely running inside the GPU, the backcopy from device to host at the end of computeForce is no longer necessary.

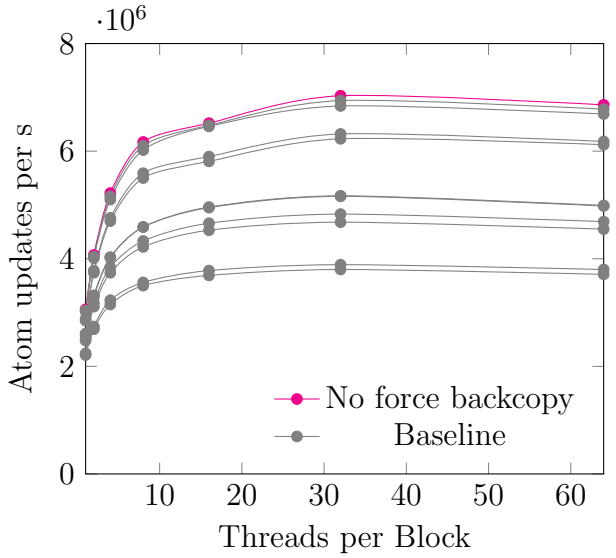


Figure 2.11: Improvement V: Force integration without backcopy on GPU

Duration	Size	Throughput	SrcMemType	DstMemType	Name	Type
363.79us	4.1199MB	11.059GB/s	Pinned	Device	[CUDA memcpy HtoD]	atom.x/y/z
5.6960us	3.0000MB	514.34GB/s	Device	-	[CUDA memset]	atom.fx/fy/fz
1.4892ms	-	-	-	-	calcforce() [273]	-

Table 2.6: Memory transfers after Improvement V

This results in a reduced number of memory transfers as depicted in table 2.6.

Overall, performance as shown in tables 2.9, 2.10 and 2.11 was improved by 11%.

Improvement VI: Neighbour list memory layout

Instead of putting all the neighbours of the first atom adjacent to all the neighbours of the 2nd atom and so on in memory, the layout can be changed in such a way that all the first neighbours are next to all the 2nd neighbours and so on.

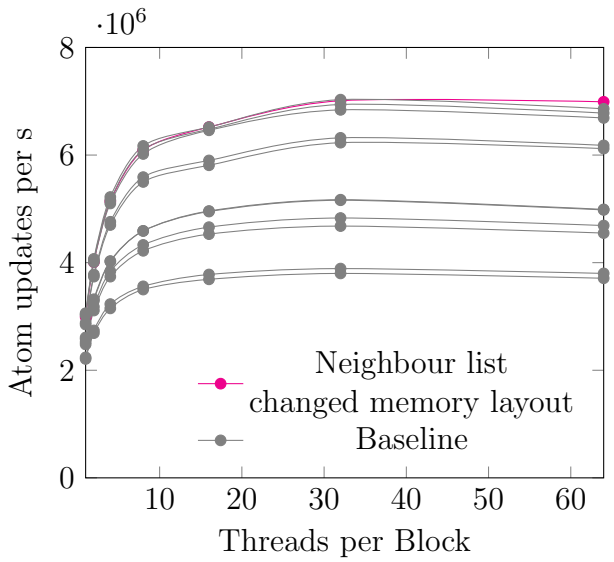


Figure 2.12: Improvement VI: Neighbour list with changed memory layout

As depicted in figure 2.12 this only results in a very minor performance improvement of at most 2%. This could be caused by the application being severely limited by memory transfers. More details will be evaluated in the next chapter.

Final performance comparisons

The first comparison is about the current implementation of MD-Bench from improvement VI between an RTX 2080 Ti and A100.

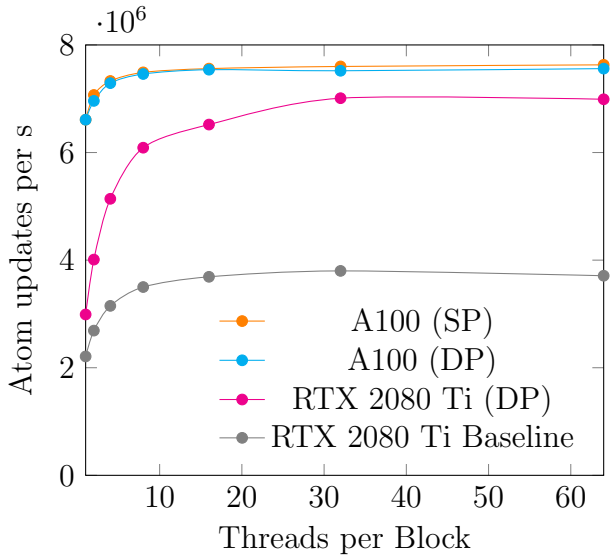


Figure 2.13: Comparison RTX 2080 Ti vs A100

As shown in figure 2.13, the A100 is memory limited much earlier than the RTX 2080 Ti which is due to the vastly higher double precision performance (10 TFLOPs vs 0.40 TFLOPs). This is also why the single precision performance is only marginally higher. In general, the A100 is about 8% faster in this comparison.

The next comparison is between the CPU implementation of miniMD running on an Intel Xeon Platinum 8360Y of FAU's Fritz cluster and the current implementation of MD-Bench on an A100.

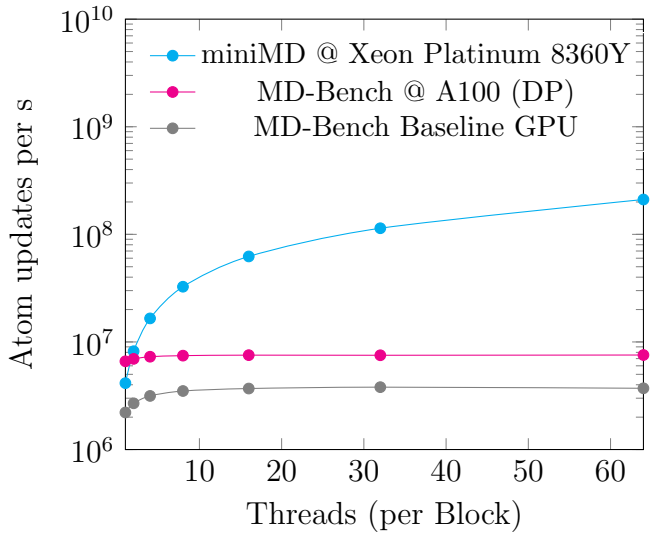


Figure 2.14: Comparison A100 vs Xeon Platinum 8360Y

The Xeon Platinum 8360Y has 2304KB of L1-Cache and 36MB of L2-Cache whereas the A100 has 192KB of L1-Cache and 40MB of L2-Cache. Detailed measurements of the CPU version of miniMD are out of the scope of this report but it could be possible that the whole miniMD program fits into L1-Cache which would explain the big lead in performance as depicted in figure 2.14.

And, of course, the atom reneighbouring algorithm hasn't been converted to a CUDA kernel yet which makes memory transfers between host and device necessary. Thus, performance of MD-Bench is limited by the transfer rate between CPU and GPU.

Taking a look at the execution time of just the computeForce function of miniMD as well as MD-Bench

Threads (per Block)	**1**	**2**	**4**	**8**	**16**	**32**	**64**
Duration miniMD (Xeon 8360Y)	27.75s	13.81s	6.61s	3.30s	1.64s	0.82s	0.43s
Duration MD-Bench (A100)	2.56µs	1.54µs	0.90µs	0.52µs	0.33µs	0.23µs	0.14µs

Table 2.7: Kernel execution time

confirms once again that the overall MD-Bench performance is limited by memory throughput as the GPU kernel is 10^6-times quicker. To alleviate the memory limitation, the reneighbouring algorithm should be converted to a CUDA kernel in the future as well.

Measurements for MD-Bench depicted in table 2.7 were done via
$ nsys profile --stats=true ./MDBench-NVCC -n 2000.

Detailed kernel measurements

For metric measurements of this chapter, the NVIDIA Nsight™ Compute tool is used. This is an interactive kernel profiler for CUDA applications. It provides detailed performance metrics via a user- and command line interface. The tool was instructed via `$ ncu --set full ./MDBench-NVCC -n 200` for 1 up to 64 threads per block on a Nvidia A100 GPU to measure the `computeForce` kernel.

Resource Utilization

Nvidia Nsight Compute provides a high-level overview for compute and memory resources of the GPU. The measured values report the achieved percentage of utilization with respect to the theoretical maximum.

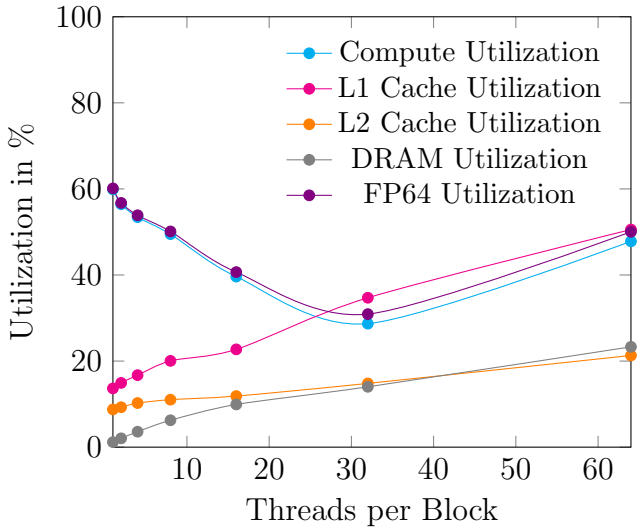


Figure 3.1: Resource utilization

Figure 3.1 shows that no compute or memory resource is at its limit regarding utilization. The decrease of compute and FP64 utilization from 1 to 32 threads per block can be explained by the fact that block execution always happens on a warp-size level, which is multiple of 32 threads.

Memory utilization is constantly increasing because L1- and L2-Cache can be used as shared memory accorss threads within a streaming multiprocessor. So it only makes sense that the utilization increases with more threads.

FLOPs/s

Measuring the number of floating point (double precision) operations per second indicates how well the kernel scales with the number of threads.

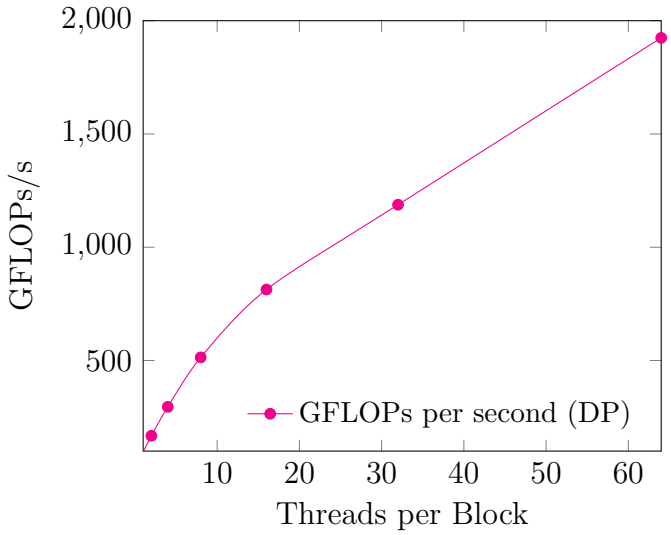


Figure 3.2: Floating point operations per second

As figure 3.2 and also table 2.7 already indicated, the computeForce kernel scales well with more threads.

Memory Transfers

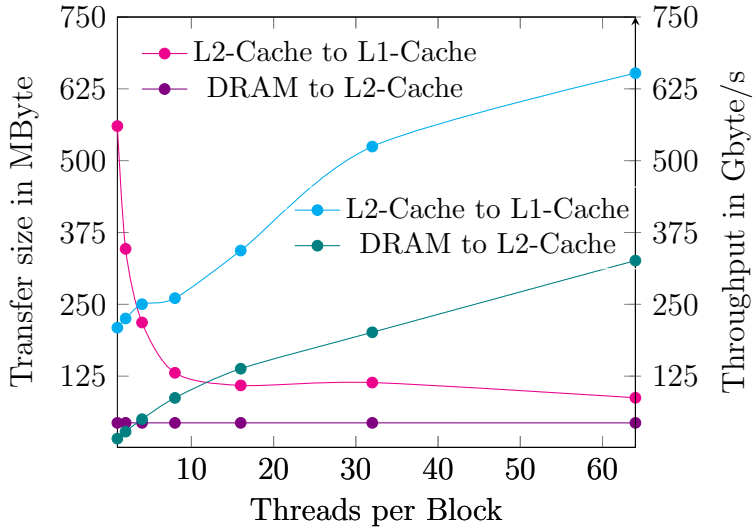


Figure 3.3: Memory transfers

Figure 3.3 shows on one hand that the transfer size between L2- and L1-Cache decreases with increasing amount of threads per block and on the other hand that the throughput increases.

This can be explained similarly to the memory utilization in figure 3.1: With more threads allocated for a streaming multiprocessor (SM) the amount of data that needs to be fetched from L2-Cache is less than if the same number of threads is spread out on more SMs (less threads per block). Thus, the memory throughput increases as more threads on an SM are accessing the shared L1- or L2-Cache.

The transfer size from DRAM to L2-Cache stays the same at around 40MB no matter how many threads per blocks. This indicates, that the whole application can fit into L2-Cache, which is 40MB in size in the case of an A100.

Conclusion

Bibliography